Tarea 2: Sentencias de control

Tomas Santana

Octubre 2026

1 Descripción

Resolver cuatro problemas en C++ usando condicionales, ciclos, contadores, acumuladores, conmutadores e interruptores, la sentencia `switch`, menús y funciones. Cada ejercicio es un programa independiente.

#	Archivo	Practica
0	`00_ejemplo.cpp`	Ejemplo resuelto: no se entrega.
1	`01_ventas.cpp`	`while` con centinela, contadores, acumuladores, `else if`, máximo.
2	`02_fichas.cpp`	Conmutador (turnos), interruptor, condiciones compuestas, `for` dentro de `while`.
3	`03_expendedora.cpp`	Menú con `do-while` y `switch`, estado que se conserva entre opciones, funciones.
4	`04_numeros.cpp`	Funciones que llaman a otras funciones, ciclos con `%` y `/` sobre los dígitos.

A diferencia de la tarea 1, los programas **pueden y deben** comunicarse con el usuario: mostrar un encabezado, un menú y un mensaje antes de cada lectura. El evaluador solo lee los resultados marcados con su **etiqueta** (sección 3).

2 Evaluación

La nota tiene dos componentes: la implementación de los programas y su explicación en clase.

$$\text{Nota} = \text{Implementación} \times \text{Explicación}$$

Componente	Escala	Qué se evalúa
Implementación	0 a 20	Que los programas compilen, cumplan las reglas y produzcan los resultados correctos.
Explicación	0 a 1	Que el estudiante explique su código, prediga su comportamiento y lo modifique en clase.

> **Importante:** la explicación multiplica la nota. Una tarea perfecta (20) que el estudiante no puede explicar (0) tiene una nota de **0**. Una tarea de 16 explicada con seguridad (1) tiene una nota de **16**.

2.1 Implementación (0 a 20)

Cada ejercicio vale 5 puntos:

Criterio (por ejercicio)	Puntos
Casos de prueba: los públicos, que se entregan con la tarea, y otros que no se publican. Los puntos son proporcionales a los casos aprobados.	4
Interfaz y legibilidad: un mensaje claro antes de cada lectura, resultados con su etiqueta, nombres de variables descriptivos, indentación y comentarios. En los ejercicios 3 y 4 este punto también exige las funciones obligatorias.	1
Total por ejercicio	**5**

Un ejercicio que no compila tiene 0. Superar las pruebas públicas no garantiza superar todas las pruebas de evaluación.

2.2 Explicación (0 a 1)

En la semana 8, en clase, cada estudiante defiende **un ejercicio elegido al azar**, en unos 3 o 4 minutos, con su código abierto en VS Code. El profesor puede pedirle:

- Explicar qué hace una parte específica del código y por qué se escribió de esa manera.
- Indicar qué mostrará el programa con ciertos datos antes de ejecutarlo.
- Hacer en el momento un cambio pequeño en las reglas y demostrar que funciona.

Valor	Criterio
1	Explica con seguridad, predice la salida correctamente y hace el cambio solicitado.
0.75	Duda en algún detalle o necesita una pista pequeña.
0.5	Explica la idea general, pero no justifica sus decisiones ni logra hacer el cambio.
0.25	Reconoce partes del código, pero no explica cómo funcionan.
0	No puede explicar su propio código.

Como el ejercicio se sortea, hay que poder explicar los cuatro. Si el estudiante no se presenta, la explicación vale 0 y la nota es 0, salvo uso del comodín o retraso justificado según el sílabo.

> **Honestidad académica:** realiza la tarea por tu cuenta. **No se permite usar herramientas de IA generativa** (ChatGPT, Copilot, Gemini, Claude…) para escribir o corregir el código. El trabajo debe ser original, según lo establecido en el sílabo; debes comprender cada línea de tus programas.

3 Cómo se revisa la salida: etiquetas

Cada ejercicio tiene una lista de **etiquetas obligatorias**, por ejemplo `Total` o `Mayor`. El evaluador ignora todo lo demás que imprima tu programa y solo busca esas etiquetas.

1. **Puedes imprimir lo que quieras:** encabezados, menús, mensajes antes de cada `cin`, separadores, decoración.
2. **Cada resultado se imprime como `Etiqueta: valor`**, con la etiqueta exacta del enunciado. No importan las mayúsculas, los espacios extra ni un `$` antes del número: `Total: 106`, `TOTAL:   $106` y `total:106` son equivalentes.
3. **Las etiquetas se usan solo para los resultados.** Si una aparece en un menú o en un mensaje, el evaluador la lee como un resultado más y el caso falla.

El evaluador exige que aparezcan exactamente los resultados esperados, en el mismo orden: ni más, ni menos. Los valores con decimales se aceptan con 2 o más decimales (`21.20` , `21.2000`).

Escribe las etiquetas **sin tildes** (`Pequenas` , `Credito` , `Capicua`): así se ven bien en la consola de Windows.

Por ejemplo, en el ejercicio 1 estas líneas se comportan así:

Tu programa imprime	El evaluador
`VENTAS DEL DIA`	Lo ignora: no tiene etiqueta.
`Venta #1:`	Lo ignora: `Venta #1` no es una etiqueta del ejercicio.
`   -> mediana`	Lo ignora.
`Total:    $106`	Lee el resultado `Total: 106`.
`Subtotal: 106`	Lo ignora: `Total` debe estar al inicio de una palabra.
`Total vendido: 106`	Lo ignora: después de la etiqueta debe venir `:`.
`Monto total: 106`	Lee `Total: 106`: la etiqueta puede ir después de otras palabras.
`Ventas 5`	Lo ignora: falta el `:`. El resultado `Ventas` no se encuentra y el caso falla.
`Ingresa el Total:`	Lo lee como un resultado más (sin número) y el caso falla. No uses etiquetas en los mensajes.

4 Restricciones

- Cada ejercicio es un programa independiente en su archivo. Conserva los nombres de los archivos.
- En toda la tarea están prohibidos los arreglos, `string` (los mensajes se imprimen como texto entre comillas), el paso por referencia (`&`), los punteros y la recursividad.
- **Ejercicios 1 y 2:** sin `switch` ni funciones propias.
- **Ejercicios 3 y 4:** el menú se repite con `do-while` y las opciones se eligen con `switch`. Deben incluir las funciones obligatorias indicadas, con parámetros por valor y `return`.
- No se necesita `<cmath>`. Solo el ejercicio 1 necesita `<iomanip>`, para el promedio.
- Las entradas siempre cumplen las restricciones del enunciado.

5 Ejercicios

Cada archivo `.cpp` incluye el enunciado completo y comentarios que guían los pasos. En los ejemplos, los números escritos por el usuario aparecen después de cada mensaje.

5.1 Ejercicio 0: Suma y pares (ejemplo resuelto)

Archivo: `00_ejemplo.cpp`. **Este ejercicio ya está resuelto y no se entrega.**

Léelo antes de empezar: muestra cómo imprimir un encabezado, un mensaje antes de cada lectura y los resultados con su etiqueta, y explica qué lee el evaluador y qué ignora. También sirve para comprobar que el compilador está bien instalado: si el evaluador muestra `OK` en el ejercicio 0, todo funciona.

Se leen N enteros (N entre 0 y 100; cada número entre −1000 y 1000). Etiquetas: `Suma` y `Pares` (cuántos son pares).

```
==============================
       SUMA Y PARES
==============================
Cuantos numeros vas a ingresar? 4
Numero 1: 7
Numero 2: 2
Numero 3: -6
Numero 4: 3
------------------------------
```

```
Suma:  6
Pares: 2
```

5.2 Ejercicio 1: Ventas del día

Archivo: `01_ventas.cpp`.

Una tienda anota el monto de cada venta del día y, para terminar, escribe `0`. Cada venta se clasifica como **pequeña** (menos de $10), **mediana** (de $10 a $49) o **grande** ($50 o más).

Entrada: enteros entre 1 y 10000, uno por venta, terminados en `0`. Puede no haber ventas. Hay a lo sumo 1000 ventas.

Etiquetas obligatorias, en este orden: `Ventas` (cantidad), `Total`, `Pequenas`, `Medianas`, `Grandes`, `Mayor` (la venta mayor) y `Promedio`. El promedio es un número real; imprímelo con 2 decimales o más. Si no hubo ventas, todos los valores son 0.

La clasificación de cada venta (`-> mediana`) es decoración sugerida: no es una etiqueta.

```
==============================
        VENTAS DEL DIA
==============================
Monto de cada venta (0 para terminar)
Venta #1: 12
   -> mediana
Venta #2: 5
   -> pequena
Venta #3: 50
   -> grande
Venta #4: 9
   -> pequena
Venta #5: 30
   -> mediana
Venta #6: 0

------------------------------
Ventas:   5
Total:    $106
Pequenas: 2
Medianas: 2
Grandes:  1
Mayor:    $50
Promedio: $21.20
```

Cuidado: 106 / 5 da 21 si los dos números son `int`. El promedio correcto es 21.2.

5.3 Ejercicio 2: Juego de fichas

Archivo: `02_fichas.cpp`.

Sobre la mesa hay N fichas. Los jugadores 1 y 2 se turnan, y empieza el 1. En cada turno, el jugador toma 1, 2 o 3 fichas, sin pasar de las que quedan. **Gana quien toma la última ficha.** Si un jugador hace una jugada inválida (menos de 1, más de 3 o más fichas de las que quedan), **pierde de inmediato** y la partida termina.

Entrada: N, entre 1 y 100, y luego las jugadas, una por turno. Las jugadas son enteros cualesquiera. Siempre hay suficientes jugadas para terminar la partida; las que sobren no se leen.

Etiqueta	Cuándo se imprime
`Quedan`	Después de cada jugada válida, con las fichas que quedan.

Etiqueta	Cuándo se imprime
`Error`	`Error: jugada invalida`, si ocurre una.
`Ganador`	Al final: 1 o 2.
`Jugadas`	Al final: cuántas jugadas válidas hubo.

La línea `Mesa: | | |` es decoración sugerida, no una etiqueta.

```
==============================
        JUEGO DE FICHAS
==============================
Fichas sobre la mesa: 5

Mesa: | | | | |
Jugador 1, cuantas fichas tomas (1 a 3)? 3

Quedan: 2

Mesa: | |
Jugador 2, cuantas fichas tomas (1 a 3)? 3

Error: jugada invalida
------------------------------
Ganador: 1
Jugadas: 1
```

Con la entrada `10  3 2 3 2` deben aparecer, en orden: `Quedan: 7`, `Quedan: 5`, `Quedan: 2`, `Quedan: 0`, `Ganador: 2` y `Jugadas: 4`.

5.4 Ejercicio 3: Máquina expendedora

Archivo: `03_expendedora.cpp`.

Una máquina vende agua (producto `1`, $1), jugo (`2`, $3) y galletas (`3`, $2), y acepta billetes de $1, $5 y $10. El usuario inserta billetes, que se suman a su crédito, y con ese crédito compra. El programa muestra un menú y repite hasta que se elige `0`.

Comando	Qué hace	Qué imprime
`1 b`	Inserta el billete `b`.	`Credito: $x`, o `Error: billete invalido`.
`2 p`	Compra el producto `p`.	`Entregado: agua` (o `jugo`, `galletas`) y luego `Credito: $x`; o `Error: producto invalido`; o `Error: credito insuficiente`.
`3`	Devuelve el crédito, que queda en 0.	`Devuelto: $x`.
`0`	Apaga la máquina.	`Vendidos: n` y `Recaudado: $r`.
otro	Nada.	`Error: opcion invalida`.

- Al comprar, primero se revisa que el producto exista y después que alcance el crédito.
- Una opción inválida no lee ningún dato adicional.
- Al apagar, el crédito que quede se ignora: no se devuelve.
- `Recaudado` es la suma de los precios de los productos vendidos.

Funciones obligatorias:

- `int precio(int producto)`: devuelve el precio, o 0 si el producto no existe. Usa un `switch`.
- `bool billeteValido(int b)`: devuelve `true` si `b` es 1, 5 o 10.

> El menú puede mostrar el saldo, pero **no** con la etiqueta `Credito:`. Usa, por ejemplo, `Saldo actual: $5`. Si no, el evaluador leerá el menú como un resultado más.

Con la entrada `1 5  2 2  2 2  1 1  2 3  3  1 7  2 9  0` deben aparecer, en este orden:

```
Credito: $5
Entregado: jugo
Credito: $2
Error: credito insuficiente
Credito: $3
Entregado: galletas
Credito: $1
Devuelto: $1
Error: billete invalido
Error: producto invalido
Vendidos: 2
Recaudado: $5
```

Así puede verse una vuelta del menú:

```
============================
      MAQUINA EXPENDEDORA
============================
  Saldo actual: $5
  1. Insertar billete ($1, $5, $10)
  2. Comprar producto
       1) Agua $1   2) Jugo $3   3) Galletas $2
  3. Devolver dinero
  0. Apagar
Opcion: 2
Producto: 2

Entregado: jugo
Credito: $2
```

5.5 Ejercicio 4: Analizador de números

Archivo: `04_numeros.cpp`.

El programa muestra un menú y repite hasta que se elige `0`. En cada consulta se lee un entero `n` entre 0 y 1000000.

Comando	Qué hace	Qué imprime
`1 n`	Suma los dígitos de `n`.	`Suma de digitos: x`
`2 n`	Invierte los dígitos de `n`.	`Invertido: x`
`3 n`	¿Se lee igual al revés?	`Capicua: SI` o `Capicua: NO`
`4 n`	¿Es primo?	`Primo: SI` o `Primo: NO`
`0`	Termina.	`Consultas: k`
otro	Nada.	`Error: opcion invalida`

- Al invertir se pierden los ceros finales: `1200` da `21` y `0` da `0`.
- Un número es capicúa si es igual a su invertido: `12321` lo es; `1210` no, porque su invertido es `121`. El 0 y los números de un dígito son capicúas.

- Un número es primo si es mayor que 1 y solo es divisible entre 1 y entre sí mismo. El 0 y el 1 no son primos; el 2 sí.
- `Consultas` es la cantidad de consultas válidas (opciones 1 a 4). Una opción inválida no lee ningún dato adicional.

Funciones obligatorias: `int sumaDigitos(int n)`, `int invertir(int n)`, `bool esCapicua(int n)`, que debe usar `invertir`, y `bool esPrimo(int n)`.

Con la entrada `1 4528  2 1200  3 12321  3 1210  4 97  4 1  7  0` deben aparecer, en este orden:

```
Suma de digitos: 19
Invertido: 21
Capicua: SI
Capicua: NO
Primo: SI
Primo: NO
Error: opcion invalida
Consultas: 6
```

6 Ejecutar las pruebas públicas

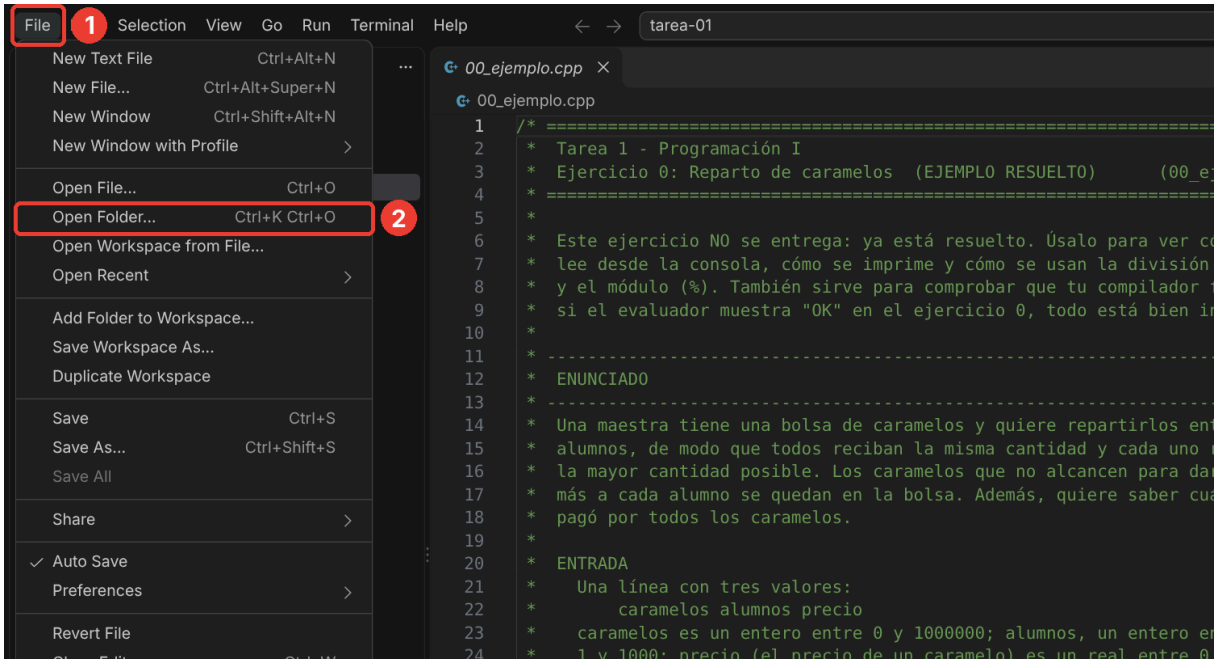
Solo se necesita VS Code y un compilador de C++17. Si todavía no los tienes, sigue primero la **guía de instalación** (`SETUP.pdf`), que explica paso a paso cómo instalar todo en Windows y en macOS.

- **Windows:** descomprime el compilador portátil `w64devkit` dentro de la carpeta de la tarea, de modo que exista la carpeta `w64devkit/bin`. Si ya lo tienes en la carpeta de la tarea 1, cópialo.
- **macOS:** instala las herramientas de desarrollo de Apple con `xcode-select --install`.

6.1 Opción 1: desde VS Code (recomendada)

Las capturas muestran VS Code en inglés, que es como viene instalado. Si tu VS Code está en español, los menús se llaman igual pero traducidos: se indican entre paréntesis.

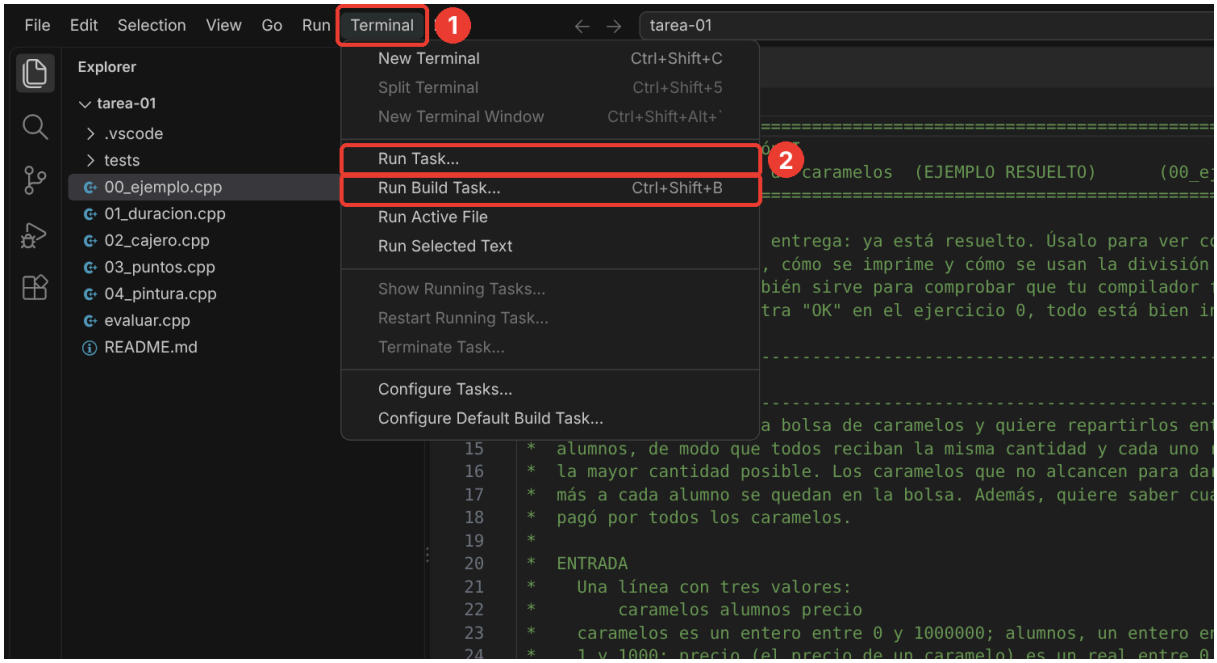
Paso 1. Abre la carpeta de la tarea. Ve a **File > Open Folder...** (*Archivo > Abrir carpeta...*) y elige la carpeta `tarea-02`, la que contiene este documento. Abre la carpeta completa, no un archivo suelto: si no, VS Code no encuentra la configuración de las pruebas.



Menú File con la opción Open Folder resaltada.

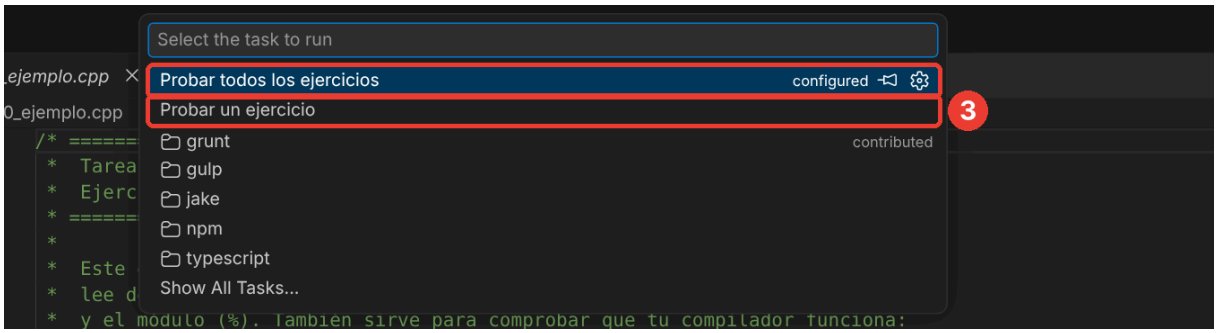
A la izquierda, en el panel **Explorer** (*Explorador*), deberías ver los archivos de la tarea y la carpeta `.vscode`.

Paso 2. Abre el menú de tareas. Ve a **Terminal > Run Task...** (*Terminal > Ejecutar tarea...*).



Menú Terminal con la opción Run Task resaltada.

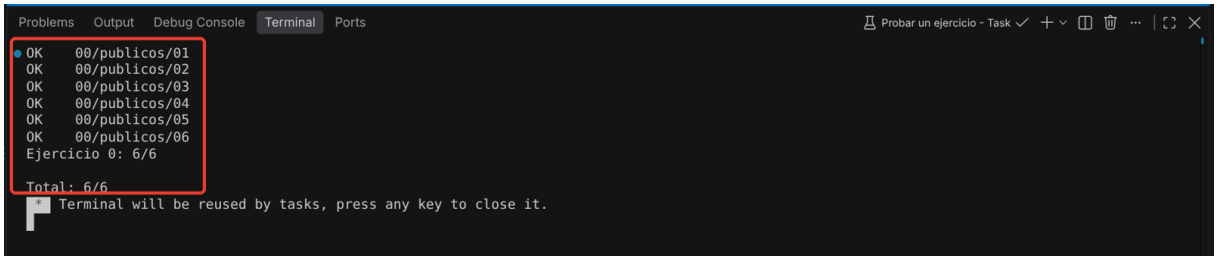
Paso 3. Elige qué probar. Aparece una lista arriba, en el centro de la ventana. Elige **Probar un ejercicio**, o **Probar todos los ejercicios** para correrlos todos a la vez. Ignora las demás opciones (grunt, gulp, npm...): vienen con VS Code y no se usan en esta tarea.



Lista de tareas con Probar un ejercicio resaltado.

Paso 4. Elige el número del ejercicio. La primera vez, elige el **0**: es el ejemplo resuelto y sirve para comprobar que todo está bien instalado.

Paso 5. Lee el resultado. Se abre la terminal en la parte de abajo. Si todo está bien instalado, el ejercicio 0 muestra `OK` en sus seis casos y `Total: 6/6`.



Terminal con el ejercicio 0 aprobado.

Para cerrar el resultado, presiona cualquier tecla en la terminal.

Atajo: `Ctrl+Shift+B` (`Cmd+Shift+B` en macOS) prueba todos los ejercicios directamente, sin pasar por los menús.

6.2 Opción 2: desde la terminal

Desde la carpeta de la tarea, compila el evaluador una sola vez:

```
g++ -std=c++17 -O2 evaluar.cpp -o evaluar
```

Luego ejecútalo. Empieza por el ejercicio 0 para comprobar que todo funciona:

```
./evaluar 0        # macOS: solo el ejemplo resuelto
./evaluar          # macOS: todos los ejercicios
./evaluar 3        # macOS: solo el ejercicio 3
.\evaluar.exe 0    # Windows: solo el ejemplo resuelto
.\evaluar.exe      # Windows: todos los ejercicios
.\evaluar.exe 3    # Windows: solo el ejercicio 3
```

Si la terminal no encuentra `g++`, indica la ruta del compilador:

```
.\evaluar.exe --compilador "w64devkit/bin/g++.exe"
```

6.3 Cómo leer el resultado

Cada caso se muestra como `OK` o `FALLO`. Cuando un caso falla, el evaluador muestra la entrada del caso, los resultados que esperaba encontrar, los que encontró, la primera diferencia y la salida completa de tu programa. Por ejemplo, así falla el ejercicio 1 cuando el promedio se calcula con división entera:

```
FALLO 01/publicos/06: salida incorrecta
   entrada: 1 2 3 4 0
   se esperaba encontrar, en este orden:
      1. Ventas: 4
      ...
      7. Promedio: 2.500000
   se encontró:
      1. Ventas: 4
      ...
      7. Promedio: 2.00
   primera diferencia: en la posición 7 se esperaba "Promedio: 2.500000"
                       y se encontró "Promedio: 2.00"
   salida completa de tu programa:
      | ==============================
      |          VENTAS DEL DIA
      | ==============================
      | Monto de cada venta (0 para terminar)
      | Venta #1:    -> pequena
      ...
      | Promedio: $2.00
```

En la salida completa no aparecen los números de la entrada: el evaluador se los pasa a tu programa desde un archivo, como si alguien los escribiera, y lo que se escribe no forma parte de la salida.

Otros mensajes posibles:

- `sobra "Credito: 0" ... (¿imprimiste una etiqueta en un menú o un mensaje?)`: una etiqueta aparece donde no es un resultado.
- `falta "..." (no se encontró ninguna etiqueta)`: el programa no imprimió ningún resultado reconocible; por ejemplo, la plantilla sin resolver.
- `tiempo excedido; ¿ciclo infinito?` o `salida demasiado larga; ¿ciclo infinito?`: el programa no terminó en 2 segundos o imprimió sin parar. Revisa la condición de tus ciclos y que la variable de la condición cambie dentro del ciclo.

Las plantillas vacías compilan, pero fallarán hasta que implementes cada solución.

7 Entrega

1. Entrega los cuatro archivos `01_ventas.cpp`, `02_fichas.cpp`, `03_expendedora.cpp` y `04_numeros.cpp`, con esos nombres, en la fecha y por el medio indicados.
2. El día de la defensa, ten la tarea abierta en VS Code y lista para ejecutarse.